

prog_datasci_5_api

April 28, 2024

1 Fonaments de Programació

1.1 Unitat 5: Adquisició de dades en Python

2 Instruccions d'ús

Aquest document és un *notebook* interactiu que intercala explicacions més aviat teòriques de conceptes de programació amb fragments de codi executables. Per aprofitar els avantatges que aporta aquest format, us recomanem que, en primer lloc, llegiu les explicacions i el codi que us proporcionem. D'aquesta manera tindreu un primer contacte amb els conceptes que hi exposem. Ara bé, **la lectura és només el principi!** Una vegada hàgiu llegit el contingut, no oblideu executar el codi proporcionat i modificar-lo per crear-ne variants que us permetin comprovar que heu entès la seva funcionalitat i explorar-ne els detalls d'implementació. Per últim, us recomanem també consultar la documentació enllaçada per explorar amb més profunditat les funcionalitats dels mòduls presentats.

Per tal de guardar possibles modificacions que feu sobre aquest notebook, us aconsellem que munteu la unitat de Drive a Google Colaboratory (colab). Per fer-ho, heu d'executar les següents instruccions:

```
[2]: from google.colab import drive  
drive.mount('/content/drive')
```

Mounted at /content/drive

```
[3]: %cd /content/drive/MyDrive/Colab_Notebooks/prog_datasci_5
```

/content/drive/MyDrive/Colab_Notebooks/prog_datasci_5

3 Introducció

Aquesta unitat presenta tres mètodes diferents per obtenir dades a internet: la descàrrega directa de les dades, l'ús d'APIs de tercers, i el *web crawling*. Per a aquests dos últims mètodes, veurem com implementar-los en Python.

A continuació s'inclou la taula de continguts, que podeu fer servir per navegar pel document:

1	Introducció
2	Descàrrega directa de dades
3	Petició a APIs de tercers
3.1	El concepte d'API
3.2	Peticions i respostes HTTP
3.3	JSON i XML
3.4	Ús d'API de tercers en Python
3.4.1	Accés a API manualment
3.4.2	Accés a API amb llibreries de Python
4	Obtenció de dades a partir de web crawling
4.1.	HTML
4.2	Web crawling en Python amb Scrapy
4.3	Web crawling en Python amb BeautifulSoup
5	Exercicis i preguntes teòriques
5.1	Instruccions importants
6.	Bibliografia
7.	Annex: L'API de googlemaps
# 1	Introducció

Els processos d'adquisició de dades són molt diversos. En aquesta unitat, veurem exemples d'adquisició de dades d'internet amb tres mètodes diferents:

- descàrrega directa
- petició a APIs de tercers
- *web crawling*

Pel que respecta a la interacció amb APIs de tercers, repassarem dues alternatives, la construcció manual de les peticions HTTP i l'ús de llibreries Python.

En relació amb el *web crawling*, veurem com utilitzar la llibreria [Scrapy](#) per construir un petit *web crawler* que capturi dades del nostre interès.

# 2	Descàrrega directa de dades
-----	-----------------------------

La descàrrega directa del conjunt de dades és potser el mètode més senzill d'adquisició de dades i consisteix a descarregar un fitxer amb les dades d'interès ja recopilades per algun altre analista. De fet, a la unitat anterior ja hem fet servir aquest mètode per adquirir el fitxer amb les dades sobre els personatges de còmic de Marvel. Un cop descarregat el fitxer, el procediment per carregar-lo en Python dependrà del format concret (ja hem vist un exemple de càrrega de dades des d'un fitxer .csv).

Alguns dels llocs web on podeu trobar conjunts de dades a analitzar són: - [Open Data gencat](#), el portal de dades obertes de la Generalitat. - [datos.gov.es](#), el catàleg de conjunts de dades del Govern d'Espanya. - [European Data Sources](#), el portal de dades obertes de la Unió Europea. - [Mark Newman network datasets](#), conjunts de dades en forma de xarxa recopilats per Mark Newman. - [Stanford Large Network Dataset Collection](#), un altre recopilatori de conjunts de dades en forma de xarxa, en aquest cas creat per Jure Leskovec. - [SecRepo.com](#), dades relacionades amb la seguretat. - [AWS Public Datasets](#), conjunts de dades recopilades i hostatjades per Amazon. - [UC Irvine Machine Learning Repository](#), dades recopilades per un grup de recerca de la Universitat de Califòrnia, Irvine. - El [repositori de Five Thirty Eight](#), que recull dades utilitzades a articles de la publicació i que ja hem vist a la unitat anterior.

3 Petició a APIs de tercers

En aquest apartat definirem què és una API, descriurem les peticions i respostes HTTP que s'utilitzen per interactuar amb les API web i, finalment, presentarem els dos formats de representació de dades més utilitzades per les API web.

3.1 El concepte d'API

Una **API** (de l'anglès, *Application Programming Interface*) és un conjunt de mètodes de comunicació entre diversos components de programari.

Les API faciliten el treball d'integració de programes, ja que permeten oferir una interfície clara i ben especificada amb la qual interactuar amb una aplicació, amagant-ne els detalls de la implementació i exposant únicament funcions específiques d'interès.

La definició d'API és molt genèrica i podem trobar diverses API en molts contextos. En aquesta unitat, ens centrarem en l'ús de les API web per a l'adquisició de dades de serveis de tercers. Les **API web** es defineixen habitualment com un conjunt de peticions HTTP juntament amb l'especificació de l'estructura de les dades de les respostes, normalment en format JSON o XML.

L'ús de les API web està molt estès actualment per interactuar amb grans proveïdors de serveis a internet. Alguns exemples d'API populars són les de [Google maps](#), [YouTube](#), [Spotify](#), [Twitter](#) o [Facebook](#).

Diem que una API és RESTful (o, de vegades, simplement REST) quan compleix un conjunt de característiques, entre les quals destaca que no manté l'estat entre peticions. És a dir, tota la informació necessària per respondre una petició es troba en la petició individual (i no depèn de cap estat emmagatzemat pel servidor).

3.2 3.2 Peticions i respostes HTTP

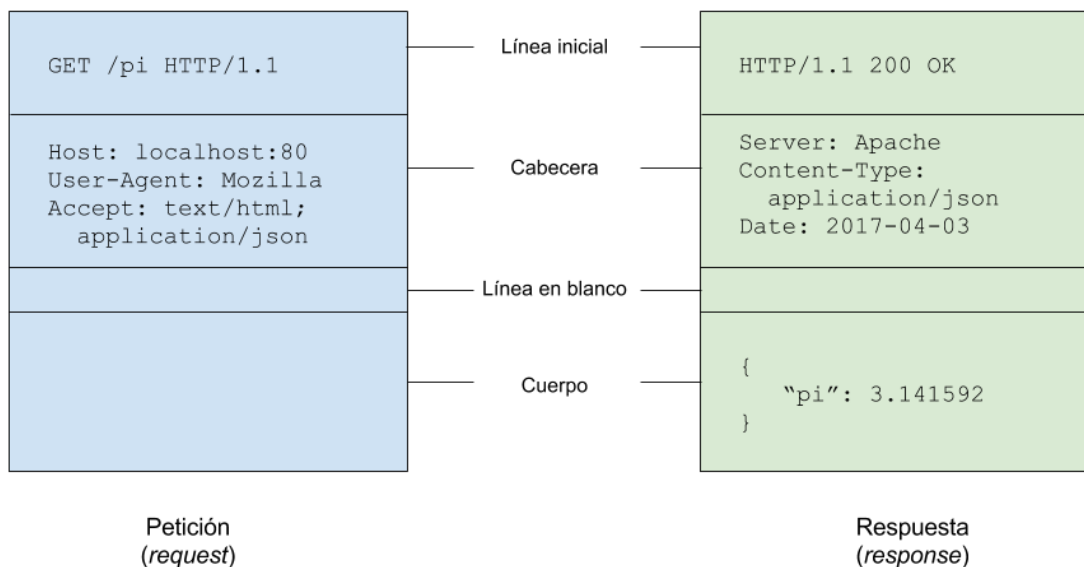
Per interactuar amb una web API farem una petició HTTP. Al seu torn, el servidor ens respondrà amb un missatge de resposta HTTP. Les peticions i respostes HTTP s'estructuren en tres parts:

- Una línia inicial de petició, que inclou l'acció que cal realitzar (el mètode de la petició) i la URL del recurs, a les peticions; i el codi d'estat i el missatge associat, en el cas de les respostes.
- La capçalera, que inclou metadades amb diverses finalitats, per exemple, per descriure'n el contingut, fer-ne l'autenticació o controlar les galetes (cookies).
- Una línia en blanc que separa la capçalera del cos.
- El cos, que pot estar buit o contenir-hi dades.

En la imatge següent es mostra un exemple simplificat d'una petició i una resposta HTTP:

```
[4]: # Carreguem la llibreria per mostrar les imatges
from IPython import display
display.Image("img/HTTP_request_response.png")
```

[4]:



La línia inicial de les respostes HTTP conté el codi d'estat, un nombre enter de tres xifres que informa sobre l'intent d'entendre i processar la petició HTTP. El primer dígit del nombre defineix el tipus de resposta. Actualment, existeixen cinc tipus reconeguts:

- 1xx: informacional.
- 2xx: èxit.
- 3xx: redirecció.
- 4xx: error del client.
- 5xx: error del servidor.

Així, quan naveguem per internet, normalment les nostres peticions es responen correctament,

retornant el codi d'estat 200. De tant en tant ens trobem també amb errors del client. Per exemple, l'error 404 ens indica que la nostra màquina ha estat capaç de comunicar-se amb el servidor, però que el recurs que hem sol·licitat no existeix. Podem forçar aquest error accedint a URL no vàlides.

Les accions o mètodes més usats en interacció amb API web són:

- GET: permet obtenir informació del recurs especificat. Per exemple, quan s'introdueix un URL al navegador, el navegador envia una sol·licitud GET al servidor per obtenir la pàgina web corresponent.
- POST: permet enviar dades al recurs especificat. Per exemple, quan s'envia un formulari a una pàgina web, les dades del formulari s'envien al servidor utilitzant una sol·licitud POST.
- PUT: carrega dades actualitzant les ja existents al recurs especificat. Per exemple, si es treballa amb una API RESTful, es pot utilitzar una sol·licitud PUT per actualitzar un registre en una base de dades.
- DELETE: elimina informació del recurs especificat. Per exemple, en treballar amb una API RESTful, es pot utilitzar una sol·licitud DELETE per eliminar un registre d'una base de dades.

3.3. JSON i XML

Dos dels formats més habituals per incloure dades a les respostes de les API web són JSON i XML. Tots dos formats tenen diverses propietats en comú. En primer lloc, van ser dissenyats per ser llegits tant per humans com per ordinadors, la qual cosa els fa ideals en aquest context. En segon lloc, tots dos incorporen informació sobre l'estructura de les dades que codifiquen. Finalment, tots dos emmagatzemen les dades en text clar. No obstant això, tots dos presenten múltiples diferències.

El format **XML** (de l'anglès, Extensible Markup Language) és un llenguatge de marques que utilitza un conjunt d'etiquetes no predefinit. Els documents XML tenen un únic element arrel del qual poden penjar altres elements. Els elements es delimiten amb una etiqueta inicial i una etiqueta final. Vegem un exemple d'un document XML senzill:

```
<persona>
  <nombre>Yann</nombre>
  <cognoms>
    <cognom1>LeCun</cognom1>
    <cognom2>-</cognom2>
  </cognoms>
  <edat>56</edat>
</persona>
```

El format **JSON** (de l'anglès, Javascript Object Notation) és un subconjunt de la notació d'objectes Javascript. JSON es basa en dues estructures de dades, l'array i l'objecte, que serien equivalents a les llistes i diccionaris de Python que ja hem introduït.

Així, un array JSON és una llista ordenada de zero o de més valors, per exemple:

```
["data", "science", "course"]
```

En aquest cas, l'array està format per cadenes de caràcters.

Un objecte JSON és una col·lecció no ordenada de parells de clau i valor. Per exemple:

```
{
  "course": "Data Science",
```

```
"year": 2017
}
```

Vegem un exemple de les dades que hem representat anteriorment en XML, fent servir ara el format JSON:

```
{
  "nombre": "Yann",
  "cognoms": {
    "cognom1": "LeCun",
    "cognom2": "-"
  },
  "edat" : 56
}
```

En aquest cas, hem fet servir un objecte amb tres claus: la primera té com a valor una cadena de caràcters, la segona té com a valor un altre objecte i la tercera té com a valor un enter.

La llibreria json de Python ens ofereix algunes funcions molt útils per a treballar en aquest format. Per exemple, podem obtenir la representació JSON d'objectes Python o crear objectes Python a partir de la seva representació en JSON.

```
[5]: # Construïm un diccionari d'exemple i mostrem el tipus de dades i el contingut
      ↪ de la variable.
diccionari_exemple = {"nom": "Yann", "cognoms": {"cognom1": "LeCun", "cognom2":
      ↪ "-"}, "edat": 56}
print(type(diccionari_exemple))
print(diccionari_exemple)

# Construïm una llista d'exemple i mostrem el tipus de dades i el contingut de
      ↪ la variable.
llista_exemple = [1, 2, 3]
print(type(llista_exemple))
print(llista_exemple)
```

```
<class 'dict'>
{'nom': 'Yann', 'cognoms': {'cognom1': 'LeCun', 'cognom2': '-'}, 'edat': 56}
<class 'list'>
[1, 2, 3]
```

```
[6]: # Importem la llibreria json.
import json

# Mostrem la representació JSON del diccionari
# mitjançant la funció dumps()
json_dict = json.dumps(diccionari_exemple)
print(type(json_dict))
print(json_dict)
```

```
# Mostrem la representació JSON de la llista
# mitjançant la funció dumps()
json_list = json.dumps(llista_exemple)
print(type(json_list))
print(json_list)
```

```
<class 'str'>
{"nom": "Yann", "cognoms": {"cognom1": "LeCun", "cognom2": "-"}, "edat": 56}
<class 'str'>
[1, 2, 3]
```

Fixeu-vos que, en ambdós casos, obtenim una cadena de caràcters que ens representa, en format JSON, els objectes Python. Aquest procés es coneix com a **serialitzar** l'objecte.

També podem fer el procés invers (conegut com a **desserialitzar**), creant objectes Python (per exemple, llistes o diccionaris) a partir de cadenes de text en format JSON.

```
[7]: # Desserialitzem la cadena 'json_dict'
# mitjançant la funció loads()
diccionario_ejemplo2 = json.loads(json_dict)
print(type(diccionario_ejemplo2))
print(diccionario_ejemplo2)

# Desserialitzem la cadena 'json_list'
# mitjançant la funció loads()
lista_ejemplo2 = json.loads(json_list)
print(type(lista_ejemplo2))
print(lista_ejemplo2)
```

```
<class 'dict'>
{'nom': 'Yann', 'cognoms': {'cognom1': 'LeCun', 'cognom2': '-'}, 'edat': 56}
<class 'list'>
[1, 2, 3]
```

Per millorar la llegibilitat de les dades que obtindrem de les APIs, definirem una funció que mostrarà cadenes JSON per pantalla formatades per millorar-ne la lectura. La funció acceptarà tant cadenes de caràcters amb contingut JSON com objectes Python, i mostrarà el contingut per pantalla.

A més, la funció rebrà un paràmetre opcional que ens permetrà indicar el nombre màxim de línies que cal mostrar. Així, podrem fer servir la funció per a visualitzar les primeres línies d'un JSON llarg, sense haver de mostrar el JSON complet per pantalla.

```
[8]: # Defineix la funció 'json_print', que té un paràmetre obligatori 'json_data' i
      ↪ un paràmetre opcional 'limit'
# i no torna cap valor.
# La funció mostra per pantalla el contingut de la variable 'json_data' en
      ↪ format JSON, limitant el nombre
# de línies per mostrar si s'inclou el paràmetre 'limit'.
def json_print (json_data, limit = None):
```

```

if isinstance(json_data, (str)):
    json_data = json.loads(json_data)
    nice = json.dumps(json_data, sort_keys=True, indent=3, separators=(',', ':↵
↵'))
    print("\n".join(nice.split("\n")[0:limit]))
    if limit is not None:
        print("[...]")

```

Vegem un exemple del resultat d'utilitzar la funció que acabem de definir.

```

[9]: # Mostra el valor de la variable 'json_exemple' amb la funció 'print'.
    json_exemple = '{"nom": "Yann", "cognoms": {"cognom1": "LeCun", "cognom2": ↵
↵"}, "edat": 56}'
    print(json_exemple)

```

```

{"nom": "Yann", "cognoms": {"cognom1": "LeCun", "cognom2": "-"}, "edat": 56}

```

```

[10]: # Mostra el valor de la variable 'json_exemple' amb la funció 'json_print' que ↵
↵acabem de definir.
    json_print(json_exemple)

```

```

{
  "cognoms": {
    "cognom1": "LeCun",
    "cognom2": "-"
  },
  "edat": 56,
  "nom": "Yann"
}

```

```

[11]: # Mostrem únicament les tres primeres línies.
    json_print(json_exemple, 3)

```

```

{
  "cognoms": {
    "cognom1": "LeCun",
    [...]

```

3.4 Ús d'API de tercers en Python

3.4.1. Accés a API manualment

Podem utilitzar la llibreria de Python [Requests](#) per a realitzar peticions als webs API de manera manual. Per fer-ho, haurem d'accedir a la documentació de l'API amb la qual vulguem actuar, construir manualment les peticions per obtenir la informació desitjada i processar també manualment la resposta rebuda.

Vegem un exemple de petició HTTP a una API pública. El lloc <http://postcodes.io/> ofereix una API de geolocalització sobre codis postals al Regne Unit. Llegint la documentació, podem veure que

té un mètode GET amb la URL `http://api.postcodes.io/postcodes/:código-postal`, que ens retorna informació del codi postal especificat.

```
[12]: # Importem la llibreria.
import requests

# Fem una petició get a l'API, preguntant sobre el codi postal "E98 1TT".
# Fixeu-vos que el caràcter espai es codifica com a% 20 a la URL.
response = requests.get('http://api.postcodes.io/postcodes/E98%201TT')

# Mostrem la resposta rebuda.
print("Codi d'estat de la resposta: ", response.status_code, "\n")
print("Capçalera de la resposta: ")
json_print(dict(response.headers))
print("\nCcos de la resposta: ")
json_print(response.text)
```

Codi d'estat de la resposta: 200

Capçalera de la resposta:

```
{
  "CF-Cache-Status": "MISS",
  "CF-RAY": "87b93e82195a62c9-ORD",
  "Connection": "keep-alive",
  "Content-Encoding": "gzip",
  "Content-Type": "application/json; charset=utf-8",
  "Date": "Sun, 28 Apr 2024 18:51:31 GMT",
  "NEL": "{\"success_fraction\":0,\"report_to\":\"cf-nel\", \"max_age\":604800}\",
  "Report-To": "{\"endpoints\": [{\"url\":\"https://a.nel.cloudflare.com/v4?s=bn0Hq7ZfAn5y8dJ03eqIkyDIuw%2BEMEQMrIv2mfDPyfkIP0eh6mbht9n6vPSnpheYG5DHWGeHyfqXCIMhbUA2g%2F9ZmIUWLP%2FStai7klqVXKR%2BvFii78qv%2FbFjfn%2FPYAfNBw%3D\"}], \"group\":\"cf-nel\", \"max_age\":604800}\",
  "Server": "cloudflare",
  "Transfer-Encoding": "chunked",
  "Vary": "Accept-Encoding",
  "access-control-allow-origin": "*",
  "etag": "W/\"428-D/azuRac8VYutBCCTAUS21zezi4\"",
  "x-gnu": "Michael J Blanchard"
}
```

Ccos de la resposta:

```
{
  "result": {
    "admin_county": null,
    "admin_district": "Tower Hamlets",
    "admin_ward": "St Katharine's & Wapping",
    "ccg": "NHS North East London",
```

```

"ced": null,
"codes": {
  "admin_county": "E999999999",
  "admin_district": "E09000030",
  "admin_ward": "E05009330",
  "ccg": "E38000255",
  "ccg_id": "A3A8R",
  "ced": "E999999999",
  "lau2": "E09000030",
  "lsoa": "E01004294",
  "msa": "E02000889",
  "nuts": "TLI42",
  "parish": "E43000220",
  "parliamentary_constituency": "E14000882",
  "parliamentary_constituency_2024": "E14001430",
  "pfa": "E23000001"
},
"country": "England",
"date_of_introduction": "200006",
"eastings": 534427,
"european_electoral_region": "London",
"incode": "1TT",
"latitude": 51.508024,
"longitude": -0.064393,
"lsoa": "Tower Hamlets 026B",
"msa": "Tower Hamlets 026",
"nhs_ha": "London",
"northings": 180564,
"nuts": "Tower Hamlets",
"outcode": "E98",
"parish": "Tower Hamlets, unparished area",
"parliamentary_constituency": "Poplar and Limehouse",
"parliamentary_constituency_2024": "Poplar and Limehouse",
"pfa": "Metropolitan Police",
"postcode": "E98 1TT",
"primary_care_trust": "Tower Hamlets",
"quality": 1,
"region": "London"
},
"status": 200
}

```

Com podem veure, l'estat de la resposta és 200, la qual cosa [ens indica](#) que la petició s'ha processat correctament. Entre d'altres camps, la capçalera de la resposta inclou el tipus de contingut que trobarem al cos, que serà un text en format JSON. Finalment, el cos de la resposta inclou dades sobre el codi postal consultat. Per exemple, podem veure que correspon a la nació d'Anglaterra (concretament, a la ciutat de Londres).

Fixeu-vos que podem visualitzar també la resposta accedint a la [mateixa URL](#) amb un navegador

web. En aquest cas, es poden instal·lar extensions específiques que gestionin la visualització millorada del JSON retornat (per exemple, [JSONView](#) per Chrome o Firefox).

3.4.2 Accés a API amb llibreries de Python

Encara que podríem fer servir aquest mètode per interactuar amb qualsevol API HTTP, la veritat és que quan la complexitat de les funcions disponibles incrementa (per exemple, en incloure autenticació) pot no resultar gaire pràctic. Quan vulguem accedir a APIs populars, normalment trobarem que ja existeixen llibreries de Python dissenyades per interactuar amb aquestes API, de manera que podrem obtenir dades sense necessitat de gestionar les peticions HTTP manualment.

Per exemple, Spotify, la famosa plataforma de música, té una API que permet realitzar tasques com buscar i obtenir informació sobre artistes, àlbums, pistes, llistes de reproducció, obtenir recomanacions de música, accedir a dades d'àudio, entre altres coses. Per interaccionar amb aquesta API, utilitzarem la llibreria de Python [Spotipy](#).

Autenticació amb l'API de Spotify Per utilitzar l'API de Spotify, necessiteu obtenir credencials d'autenticació. Per això, us haureu de registrar com a desenvolupador al lloc web de Spotify i crear una aplicació. Després de crear-la, obtindreu un *Client ID* i un *Client Secret* que s'utilitzaran per autenticar-se a l'API.

En aquesta secció, descriurem com obtenir credencials per accedir a l'API de Spotify.

Per començar, cal disposar d'un compte a Spotify (gratuït o premium). Podeu utilitzar el vostre compte personal, si ja en disposeu d'un, per a sol·licitar els permisos de desenvolupador que ens permetran interactuar amb l'API. En cas contrari (o si preferiu no fer servir vostre compte personal), podeu crear-vos un compte nou.

El procés per a crear un nou compte és molt senzill: 1. Accedir a [Spotify] (<https://open.spotify.com/>). 2. Prémer sobre *Sign up* i seguir les indicacions per completar el registre.

Després, caldrà crear una nova App i obtenir les credencials per connectar-se a la API. Per fer-ho, cal seguir els següents passos explicats al [tutorial](#):

1. Accediu al [panell de desenvolupadors de Spotify](#).
2. Accediu al dashboard de desenvolupadors (<https://developer.spotify.com/dashboard>). En aquest pas és possible que us faci verificar el correu electrònic.

Creació App 1. Cliqueu sobre *Create App* per a crear una nova App. 2. Ompliu el formulari amb els detalls de l'aplicació. En concret, necessitareu proporcionar com a mínim els camps: * *App name* * *Application description* * *Redirect URL* * *APIs used* Poseu el nom i la descripció que vulgueu per l'App. En l'opció de *Redirect URL* poseu: <http://localhost:3000>. Per acabar marqueu la opció *Web API*.

3. Accepteu els termes i les condicions i cliqueu *Save*.

Tokens d'accés

A continuació, s'ha de demanar un *token* d'accés. 1. En el panell de l'App que acabeu de crear, aneu a l'apartat de *Settings*. Allà trobareu el *Client ID* i el *Client Secret* (aquest últim apareix al clicar sobre *View client secret*).

Quan tingueu les credencials, ja podeu començar a utilitzar l'API de Spotify a Python.

La llibreria Spotipy [Spotipy](#) ens permet interactuar amb l'API de Spotify d'una manera senzilla, ja que encapsula els mètodes HTTP de l'API en mètodes de Python, que poden ser cridats directament. Trobareu la documentació de la llibreria en el següent [enllaç](#).

```
[13]: # Primer de tot, hem d'instalar la llibreria spotipy
!pip install spotipy
```

```
Collecting spotipy
  Downloading spotipy-2.23.0-py3-none-any.whl (29 kB)
Collecting redis>=3.5.3 (from spotipy)
  Downloading redis-5.0.4-py3-none-any.whl (251 kB)
    252.0/252.0

kB 2.8 MB/s eta 0:00:00
Requirement already satisfied: requests>=2.25.0 in
/usr/local/lib/python3.10/dist-packages (from spotipy) (2.31.0)
Requirement already satisfied: six>=1.15.0 in /usr/local/lib/python3.10/dist-
packages (from spotipy) (1.16.0)
Requirement already satisfied: urllib3>=1.26.0 in
/usr/local/lib/python3.10/dist-packages (from spotipy) (2.0.7)
Requirement already satisfied: async-timeout>=4.0.3 in
/usr/local/lib/python3.10/dist-packages (from redis>=3.5.3->spotipy) (4.0.3)
Requirement already satisfied: charset-normalizer<4,>=2 in
/usr/local/lib/python3.10/dist-packages (from requests>=2.25.0->spotipy) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-
packages (from requests>=2.25.0->spotipy) (3.7)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.10/dist-packages (from requests>=2.25.0->spotipy)
(2024.2.2)
Installing collected packages: redis, spotipy
Successfully installed redis-5.0.4 spotipy-2.23.0
```

```
[14]: # Importem la llibreria spotipy
import spotipy
from spotipy.oauth2 import SpotifyClientCredentials

# IMPORTANT: És necessari incloure les credencials d'accés que hagueu obtingut
# per executar l'exemple
your_client_id = ''
your_client_secret = ''

# Ens connectem a l'API de Spotify amb les nostres credencials
client_credentials_manager = SpotifyClientCredentials(client_id=your_client_id,
↪client_secret=your_client_secret)
sp = spotipy.Spotify(client_credentials_manager=client_credentials_manager)
```

```
# Buscarem informació sobre un artista
artist_name = 'Taylor Swift'
results = sp.search(q='artist:' + artist_name, type='artist')
artists = results['artists']['items']

if artists:
    artist = artists[0]
    print("Nom de l'artista:", artist['name'])
else:
    print("No s'ha trobat l'artista.")
```

Nom de l'artista: Taylor Swift

Fixeu-vos que, en aquest cas, no hem hagut de gestionar les peticions HTTP manualment: la llibreria ho ha fet per nosaltres de forma transparent.

A més, les funcions de la llibreria ens tornen directament objectes Python, que poden ser usats com qualsevol altre. Per exemple, podem seleccionar només una part de les respostes de les APIs segons el nostre interès (en l'exemple anterior, hem seleccionat l'identificador i el nom d'usuari directament usant l'objecte `artist`). Vegem alguns exemples més d'atributs que hem recuperat de l'artista:

```
[15]: # Mostrem alguns atributs de l'artista
print("El número de seguidors és: {}".format(artist['followers']['total']))
print("El gènere és: {}".format(artist['genres'][0]))
```

El número de seguidors és: 108607457

El gènere és: pop

4 Obtenció de dades a partir de *web crawling*

De vegades ens interessarà capturar dades que es troben a internet però per les quals no existeix una API que ens permeti accedir-hi de forma estructurada. En aquests casos, una alternativa és programar una aranya (en anglès, un *web crawler*), un programa que analitza pàgines web de forma automàtica a la recerca del contingut d'interès.

El procediment essencial d'un *web crawler* consisteix a explorar una determinada pàgina web a la recerca de dades d'interès, que s'emmagatzemaran per al seu ús posterior, i enllaços a altres pàgines web d'interès, que seran explorades posteriorment pel mateix *crawler*, a la recerca de noves dades d'interès i noves pàgines.

Per obtenir tant les dades com els enllaços d'interès, el *web crawler* utilitza un analitzador sintàctic (en anglès, *parser*), que processa l'HTML de la pàgina web i n'extreu les dades.

3.3 4.1 HTML

El format **HTML** (de l'anglès, *Hypertext Markup Language*) és el llenguatge de marques estàndard per descriure la presentació de pàgines web. De la mateixa manera que XML, utilitza (majoritàri-

ament) una etiqueta inicial i una final per indicar elements. A diferència d'XML, les etiquetes es troben prefixades per un estàndard.

A més de senyalitzar l'inici i el final d'un element, les etiquetes HTML poden incloure atributs, que permeten proporcionar informació addicional sobre els elements.

Vegem un exemple d'un document HTML senzill:

```
<html>
  <head>
    <title>El título de la página</title>
  </head>
  <body>
    <div class="clase1" id ="id1">
      <p> Un texto </p>
    </div>
    <div class="clase1" id ="id2">
      <p> Otro texto </p>
    </div>
  </body>
</html>
```

4.2 Web crawling en Python amb Scrapy

[Scrapy](#) és una llibreria de Python que proveeix d'un *framework* per a l'extracció de dades de pàgines web. Scrapy és molt complet i disposa de múltiples funcionalitats, però en veurem un exemple senzill d'ús.

Suposem que volem obtenir un llistat dels àmbits de les titulacions que ofereix la UOC. La UOC no ofereix una API amb aquesta informació, però sí que podem trobar-la a la pàgina <https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>. De totes maneres, no volem anar copiant manualment els noms de tots els àmbits per obtenir el llistat d'interès, per la qual cosa desenvoluparem un petit *crawler* que obtingui aquestes dades per nosaltres.

Ja tenim identificada la URL que volem explorar (<https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>), així que només caldrà identificar on es troben les dades d'interès dins de la pàgina. Per fer-ho, en primer lloc ens fixarem en algun títol de grau que aparegui a la pàgina, per exemple, “Estudios de Artes y Humanidades” o “Estudios de Ciencias de la Información y de la Comunicación”. Seguidament accedirem al codi font de la pàgina (podem fer servir la combinació de tecles CTRL + u als navegadors Firefox o Chrome) i buscarem els noms dels graus que hem vist anteriorment:

```
<h3 id="card-title-1682057768418" class="h3 card-title">
  <a href="/portal/es/estudis_arees/arts_humanitats/index.html" target="_blank">
    Estudios de Artes y Humanidades
    <span class="absolute-link" aria-hidden="true">
  </span>
</a>
</h3>

<h3 id="card-title-1682057768482" class="h3 card-title">
  <a href="/portal/es/estudis_arees/ciencias_informacio_comunicacio/index.html" target="_blank">
```

```

        Estudios de Ciencias de la Información y de la Comunicación
        <span class="absolute-link" aria-hidden="true">
        </span>
    </a>
</h3>

```

Com es pot apreciar, les dades que volem recopilar (els noms dels àmbits que ofereix la UOC) es troben en forma de text dintre de l'element h3 que té l'atribut classe fixat a «h3 card-title».

Per a indicar que volem seleccionar aquestes dades, utilitzarem la sintaxi XPath. En concret, utilitzarem l'expressió

```
//h3[@class="h3 card-title"]//*[text()]
```

que ens indica que volem seleccionar totes les etiquetes <h3> que tinguin com a atribut classe el valor «h3 card-title» i extreure'n el text. Com hi ha diversos elements amb aquestes característiques, especifiquem que els volem tots amb //*/. Amb això ja podem programar la nostra aranya perquè n'extregui les dades d'interès.

L'estructura d'un *crawler* amb Scrapy ve prefixada. En el nostre cas, només serà necessari definir una aranya i incloure un *parser* que extregui les dades dels àmbits i que disposi de l'URL d'inici.

Abans de veure l'estructura d'un crawler, hem d'instal·lar Scrapy a colab amb:

```
[16]: !pip install scrapy
```

```

Collecting scrapy
  Downloading Scrapy-2.11.1-py2.py3-none-any.whl (287 kB)
    287.8/287.8
kB 2.6 MB/s eta 0:00:00
Collecting Twisted>=18.9.0 (from scrapy)
  Downloading twisted-24.3.0-py3-none-any.whl (3.2 MB)
    3.2/3.2 MB
26.0 MB/s eta 0:00:00
Requirement already satisfied: cryptography>=36.0.0 in
/usr/local/lib/python3.10/dist-packages (from scrapy) (42.0.5)
Collecting cssselect>=0.9.1 (from scrapy)
  Downloading cssselect-1.2.0-py2.py3-none-any.whl (18 kB)
Collecting itemloaders>=1.0.1 (from scrapy)
  Downloading itemloaders-1.2.0-py3-none-any.whl (11 kB)
Collecting parsel>=1.5.0 (from scrapy)
  Downloading parsel-1.9.1-py2.py3-none-any.whl (17 kB)
Requirement already satisfied: pyOpenSSL>=21.0.0 in
/usr/local/lib/python3.10/dist-packages (from scrapy) (24.1.0)
Collecting queuelib>=1.4.2 (from scrapy)
  Downloading queuelib-1.6.2-py2.py3-none-any.whl (13 kB)
Collecting service-identity>=18.1.0 (from scrapy)
  Downloading service_identity-24.1.0-py3-none-any.whl (12 kB)
Collecting w3lib>=1.17.0 (from scrapy)
  Downloading w3lib-2.1.2-py3-none-any.whl (21 kB)

```

```

Collecting zope.interface>=5.1.0 (from scrapy)
  Downloading zope.interface-6.3-cp310-cp310-manylinux_2_5_x86_64.manylinux1_x86_64.manylinux_2_17_x86_64.manylinux2014_x86_64.whl (247 kB)
      247.3/247.3

kB 22.4 MB/s eta 0:00:00
Collecting protego>=0.1.15 (from scrapy)
  Downloading Protego-0.3.1-py2.py3-none-any.whl (8.5 kB)
Collecting itemadapter>=0.1.0 (from scrapy)
  Downloading itemadapter-0.8.0-py3-none-any.whl (11 kB)
Requirement already satisfied: setuptools in /usr/local/lib/python3.10/dist-packages (from scrapy) (67.7.2)
Requirement already satisfied: packaging in /usr/local/lib/python3.10/dist-packages (from scrapy) (24.0)
Collecting tldextract (from scrapy)
  Downloading tldextract-5.1.2-py3-none-any.whl (97 kB)
      97.6/97.6 kB

11.5 MB/s eta 0:00:00
Requirement already satisfied: lxml>=4.4.1 in /usr/local/lib/python3.10/dist-packages (from scrapy) (4.9.4)
Collecting PyDispatcher>=2.0.5 (from scrapy)
  Downloading PyDispatcher-2.0.7-py3-none-any.whl (12 kB)
Requirement already satisfied: cffi>=1.12 in /usr/local/lib/python3.10/dist-packages (from cryptography>=36.0.0->scrapy) (1.16.0)
Collecting jmespath>=0.9.5 (from itemloaders>=1.0.1->scrapy)
  Downloading jmespath-1.0.1-py3-none-any.whl (20 kB)
Requirement already satisfied: attrs>=19.1.0 in /usr/local/lib/python3.10/dist-packages (from service-identity>=18.1.0->scrapy) (23.2.0)
Requirement already satisfied: pyasn1 in /usr/local/lib/python3.10/dist-packages (from service-identity>=18.1.0->scrapy) (0.6.0)
Requirement already satisfied: pyasn1-modules in /usr/local/lib/python3.10/dist-packages (from service-identity>=18.1.0->scrapy) (0.4.0)
Collecting automat>=0.8.0 (from Twisted>=18.9.0->scrapy)
  Downloading Automat-22.10.0-py2.py3-none-any.whl (26 kB)
Collecting constantly>=15.1 (from Twisted>=18.9.0->scrapy)
  Downloading constantly-23.10.4-py3-none-any.whl (13 kB)
Collecting hyperlink>=17.1.1 (from Twisted>=18.9.0->scrapy)
  Downloading hyperlink-21.0.0-py2.py3-none-any.whl (74 kB)
      74.6/74.6 kB

8.8 MB/s eta 0:00:00
Collecting incremental>=22.10.0 (from Twisted>=18.9.0->scrapy)
  Downloading incremental-22.10.0-py2.py3-none-any.whl (16 kB)
Requirement already satisfied: typing-extensions>=4.2.0 in /usr/local/lib/python3.10/dist-packages (from Twisted>=18.9.0->scrapy) (4.11.0)
Requirement already satisfied: idna in /usr/local/lib/python3.10/dist-packages (from tldextract->scrapy) (3.7)
Requirement already satisfied: requests>=2.1.0 in /usr/local/lib/python3.10/dist-packages (from tldextract->scrapy) (2.31.0)

```



```

Collecting requests-file>=1.4 (from tldextract->scrapy)
  Downloading requests_file-2.0.0-py2.py3-none-any.whl (4.2 kB)
Requirement already satisfied: filelock>=3.0.8 in
/usr/local/lib/python3.10/dist-packages (from tldextract->scrapy) (3.13.4)
Requirement already satisfied: six in /usr/local/lib/python3.10/dist-packages
(from automat>=0.8.0->Twisted>=18.9.0->scrapy) (1.16.0)
Requirement already satisfied: pycparser in /usr/local/lib/python3.10/dist-
packages (from cffi>=1.12->cryptography>=36.0.0->scrapy) (2.22)
Requirement already satisfied: charset-normalizer<4,>=2 in
/usr/local/lib/python3.10/dist-packages (from
requests>=2.1.0->tldextract->scrapy) (3.3.2)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.10/dist-packages (from
requests>=2.1.0->tldextract->scrapy) (2.0.7)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.10/dist-packages (from
requests>=2.1.0->tldextract->scrapy) (2024.2.2)
Installing collected packages: PyDispatcher, incremental, zope.interface, w3lib,
queuelib, protego, jmespath, itemadapter, hyperlink, cssselect, constantly,
automat, Twisted, requests-file, parsel, tldextract, service-identity,
itemloaders, scrapy
Successfully installed PyDispatcher-2.0.7 Twisted-24.3.0 automat-22.10.0
constantly-23.10.4 cssselect-1.2.0 hyperlink-21.0.0 incremental-22.10.0
itemadapter-0.8.0 itemloaders-1.2.0 jmespath-1.0.1 parsel-1.9.1 protego-0.3.1
queuelib-1.6.2 requests-file-2.0.0 scrapy-2.11.1 service-identity-24.1.0
tldextract-5.1.2 w3lib-2.1.2 zope.interface-6.3

```

```

[17]: # Importem Scrapy.
import scrapy
from scrapy.crawler import CrawlerProcess

# Creem l'aranya.
class uoc_spider(scrapy.Spider):

    # Assignem un nom a l'aranya.
    name = "uoc_spider"

    # Indiquem l'URL que volem analitzar en primer lloc.
    start_urls = [
        "https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/
↳index.html"
    ]

    # Definim l'analitzador.
    def parse(self, response):
        # Extraiem el títol del grau.
        for ambito in response.xpath('//h3[@class="h3 card-title"]//*/text()'):

```

```

yield {
    'Ambito': ambito.extract()
}

```

Un cop definida l'aranya, llançarem el *crawler* indicant que volem que usi l'aranya *uoc_spider* que acabem de definir:

```

[18]: if __name__ == "__main__":

    # Creem un crawler:
    process = CrawlerProcess({
        'USER_AGENT': 'Mozilla/5.0 (Windows NT 10.0; WOW64) AppleWebKit/537.36_
↳ (KHTML, like Gecko) Chrome/109.0.0.0 Safari/537.36',
        'DOWNLOAD_HANDLERS': {'s3': None},
        'LOG_ENABLED': True
    })

    # Inicialitzem el crawler amb la nostra aranya:
    process.crawl(uoc_spider)

    # Llancem l'aranya:
    process.start()

```

```

INFO:scrapy.utils.log:Scrapy 2.11.1 started (bot: scrapybot)
2024-04-28 18:52:03 [scrapy.utils.log] INFO: Scrapy 2.11.1 started (bot:
scrapybot)
INFO:scrapy.utils.log:Versions: lxml 4.9.4.0, libxml2 2.10.3, cssselect 1.2.0,
parsel 1.9.1, w3lib 2.1.2, Twisted 24.3.0, Python 3.10.12 (main, Nov 20 2023,
15:14:05) [GCC 11.4.0], pyOpenSSL 24.1.0 (OpenSSL 3.2.1 30 Jan 2024),
cryptography 42.0.5, Platform Linux-6.1.58+-x86_64-with-glibc2.35
2024-04-28 18:52:03 [scrapy.utils.log] INFO: Versions: lxml 4.9.4.0, libxml2
2.10.3, cssselect 1.2.0, parsel 1.9.1, w3lib 2.1.2, Twisted 24.3.0, Python
3.10.12 (main, Nov 20 2023, 15:14:05) [GCC 11.4.0], pyOpenSSL 24.1.0 (OpenSSL
3.2.1 30 Jan 2024), cryptography 42.0.5, Platform Linux-6.1.58+-x86_64-with-
glibc2.35
INFO:scrapy.addons:Enabled addons:
[]
2024-04-28 18:52:03 [scrapy.addons] INFO: Enabled addons:
[]
/usr/local/lib/python3.10/dist-packages/scrapy/utils/request.py:254:
ScrapyDeprecationWarning: '2.6' is a deprecated value for the
'REQUEST_FINGERPRINTER_IMPLEMENTATION' setting.

```

It is also the default value. In other words, it is normal to get this warning if you have not defined a value for the 'REQUEST_FINGERPRINTER_IMPLEMENTATION' setting. This is so for backward compatibility reasons, but it will change in a future version of Scrapy.

See the documentation of the 'REQUEST_FINGERPRINTER_IMPLEMENTATION' setting for information on how to handle this deprecation.

```
    return cls(crawler)
DEBUG:scrapy.utils.log:Using reactor: twisted.internet.epollreactor.EPollReactor
2024-04-28 18:52:03 [scrapy.utils.log] DEBUG: Using reactor:
twisted.internet.epollreactor.EPollReactor
INFO:scrapy.extensions.telnet:Telnet Password: 98e684974f8a3f2d
2024-04-28 18:52:03 [scrapy.extensions.telnet] INFO: Telnet Password:
98e684974f8a3f2d
INFO:scrapy.middleware:Enabled extensions:
['scrapy.extensions.corestats.CoreStats',
 'scrapy.extensions.telnet.TelnetConsole',
 'scrapy.extensions.memusage.MemoryUsage',
 'scrapy.extensions.logstats.LogStats']
2024-04-28 18:52:03 [scrapy.middleware] INFO: Enabled extensions:
['scrapy.extensions.corestats.CoreStats',
 'scrapy.extensions.telnet.TelnetConsole',
 'scrapy.extensions.memusage.MemoryUsage',
 'scrapy.extensions.logstats.LogStats']
INFO:scrapy.crawler:Overridden settings:
{'USER_AGENT': 'Mozilla/5.0 (Windows NT 10.0; WOW64) AppleWebKit/537.36 '
                '(KHTML, like Gecko) Chrome/109.0.0.0 Safari/537.36'}
2024-04-28 18:52:03 [scrapy.crawler] INFO: Overridden settings:
{'USER_AGENT': 'Mozilla/5.0 (Windows NT 10.0; WOW64) AppleWebKit/537.36 '
                '(KHTML, like Gecko) Chrome/109.0.0.0 Safari/537.36'}
INFO:scrapy.middleware:Enabled downloader middlewares:
['scrapy.downloadermiddlewares.httpauth.HttpAuthMiddleware',
 'scrapy.downloadermiddlewares.downloadtimeout.DownloadTimeoutMiddleware',
 'scrapy.downloadermiddlewares.defaultheaders.DefaultHeadersMiddleware',
 'scrapy.downloadermiddlewares.useragent.UserAgentMiddleware',
 'scrapy.downloadermiddlewares.retry.RetryMiddleware',
 'scrapy.downloadermiddlewares.redirect.MetaRefreshMiddleware',
 'scrapy.downloadermiddlewares.httpcompression.HttpCompressionMiddleware',
 'scrapy.downloadermiddlewares.redirect.RedirectMiddleware',
 'scrapy.downloadermiddlewares.cookies.CookiesMiddleware',
 'scrapy.downloadermiddlewares.httpproxy.HttpProxyMiddleware',
 'scrapy.downloadermiddlewares.stats.DownloaderStats']
2024-04-28 18:52:03 [scrapy.middleware] INFO: Enabled downloader middlewares:
['scrapy.downloadermiddlewares.httpauth.HttpAuthMiddleware',
 'scrapy.downloadermiddlewares.downloadtimeout.DownloadTimeoutMiddleware',
 'scrapy.downloadermiddlewares.defaultheaders.DefaultHeadersMiddleware',
 'scrapy.downloadermiddlewares.useragent.UserAgentMiddleware',
 'scrapy.downloadermiddlewares.retry.RetryMiddleware',
 'scrapy.downloadermiddlewares.redirect.MetaRefreshMiddleware',
 'scrapy.downloadermiddlewares.httpcompression.HttpCompressionMiddleware',
 'scrapy.downloadermiddlewares.redirect.RedirectMiddleware',
 'scrapy.downloadermiddlewares.cookies.CookiesMiddleware',
 'scrapy.downloadermiddlewares.httpproxy.HttpProxyMiddleware',
```

```

'scrapy.downloadermiddlewares.stats.DownloaderStats']
INFO:scrapy.middleware:Enabled spider middlewares:
['scrapy.spidermiddlewares.httperror.HttpErrorMiddleware',
'scrapy.spidermiddlewares.offsite.OffsiteMiddleware',
'scrapy.spidermiddlewares.referer.RefererMiddleware',
'scrapy.spidermiddlewares.urllength.UrlLengthMiddleware',
'scrapy.spidermiddlewares.depth.DepthMiddleware']
2024-04-28 18:52:03 [scrapy.middleware] INFO: Enabled spider middlewares:
['scrapy.spidermiddlewares.httperror.HttpErrorMiddleware',
'scrapy.spidermiddlewares.offsite.OffsiteMiddleware',
'scrapy.spidermiddlewares.referer.RefererMiddleware',
'scrapy.spidermiddlewares.urllength.UrlLengthMiddleware',
'scrapy.spidermiddlewares.depth.DepthMiddleware']
INFO:scrapy.middleware:Enabled item pipelines:
[]
2024-04-28 18:52:03 [scrapy.middleware] INFO: Enabled item pipelines:
[]
INFO:scrapy.core.engine:Spider opened
2024-04-28 18:52:03 [scrapy.core.engine] INFO: Spider opened
INFO:scrapy.extensions.logstats:Crawled 0 pages (at 0 pages/min), scraped 0
items (at 0 items/min)
2024-04-28 18:52:04 [scrapy.extensions.logstats] INFO: Crawled 0 pages (at 0
pages/min), scraped 0 items (at 0 items/min)
INFO:scrapy.extensions.telnet:Telnet console listening on 127.0.0.1:6023
2024-04-28 18:52:04 [scrapy.extensions.telnet] INFO: Telnet console listening on
127.0.0.1:6023
DEBUG:scrapy.core.engine:Crawled (200) <GET
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
(referer: None)
2024-04-28 18:52:04 [scrapy.core.engine] DEBUG: Crawled (200) <GET
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
(referer: None)
DEBUG:scrapy.core.scrapers:Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'Estudios de Artes y Humanidades'}
2024-04-28 18:52:04 [scrapy.core.scrapers] DEBUG: Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'Estudios de Artes y Humanidades'}
DEBUG:scrapy.core.scrapers:Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'Estudios de Ciencias de la Información y de la Comunicación'}
2024-04-28 18:52:04 [scrapy.core.scrapers] DEBUG: Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'Estudios de Ciencias de la Información y de la Comunicación'}
DEBUG:scrapy.core.scrapers:Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'Estudios de Ciencias de la Salud'}
2024-04-28 18:52:04 [scrapy.core.scrapers] DEBUG: Scraped from <200

```

```

https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'Estudios de Ciencias de la Salud'}
DEBUG:scrapy.core.scrapers:Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'Estudios de Derecho y Ciencia Política'}
2024-04-28 18:52:04 [scrapy.core.scrapers] DEBUG: Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'Estudios de Derecho y Ciencia Política'}
DEBUG:scrapy.core.scrapers:Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'Estudios de Economía y Empresa'}
2024-04-28 18:52:04 [scrapy.core.scrapers] DEBUG: Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'Estudios de Economía y Empresa'}
DEBUG:scrapy.core.scrapers:Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'Estudios de Informática, Multimedia y Telecomunicación'}
2024-04-28 18:52:04 [scrapy.core.scrapers] DEBUG: Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'Estudios de Informática, Multimedia y Telecomunicación'}
DEBUG:scrapy.core.scrapers:Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'Estudios de Psicología y Ciencias de la Educación'}
2024-04-28 18:52:04 [scrapy.core.scrapers] DEBUG: Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'Estudios de Psicología y Ciencias de la Educación'}
DEBUG:scrapy.core.scrapers:Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'Internet Interdisciplinary Institute (IN3)'}
2024-04-28 18:52:04 [scrapy.core.scrapers] DEBUG: Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'Internet Interdisciplinary Institute (IN3)'}
DEBUG:scrapy.core.scrapers:Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'eHealth Center'}
2024-04-28 18:52:04 [scrapy.core.scrapers] DEBUG: Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'eHealth Center'}
DEBUG:scrapy.core.scrapers:Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'eLearning Innovation Center'}
2024-04-28 18:52:04 [scrapy.core.scrapers] DEBUG: Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'eLearning Innovation Center'}
DEBUG:scrapy.core.scrapers:Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'Escuela de Doctorado'}
2024-04-28 18:52:04 [scrapy.core.scrapers] DEBUG: Scraped from <200

```

```

https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'Escuela de Doctorado'}
DEBUG:scrapy.core.scrapers:Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'C tedra UNESCO de Educaci n y Tecnolog a para el Cambio Social'}
2024-04-28 18:52:04 [scrapy.core.scrapers] DEBUG: Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'C tedra UNESCO de Educaci n y Tecnolog a para el Cambio Social'}
DEBUG:scrapy.core.scrapers:Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'C tedra UNESCO de Alimentaci n, Cultura y Desarrollo'}
2024-04-28 18:52:04 [scrapy.core.scrapers] DEBUG: Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'C tedra UNESCO de Alimentaci n, Cultura y Desarrollo'}
DEBUG:scrapy.core.scrapers:Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'C tedra UOC-COCEMFE para la Autonom a Personal y Salud Digital'}
2024-04-28 18:52:04 [scrapy.core.scrapers] DEBUG: Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'C tedra UOC-COCEMFE para la Autonom a Personal y Salud Digital'}
DEBUG:scrapy.core.scrapers:Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'C tedra Barcelona - UOC en Econom a Digital'}
2024-04-28 18:52:04 [scrapy.core.scrapers] DEBUG: Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'C tedra Barcelona - UOC en Econom a Digital'}
DEBUG:scrapy.core.scrapers:Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'C tedra Pau Casals de M sica y Defensa de la Paz y de los Derechos
Humanos'}
2024-04-28 18:52:04 [scrapy.core.scrapers] DEBUG: Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'C tedra Pau Casals de M sica y Defensa de la Paz y de los Derechos
Humanos'}
DEBUG:scrapy.core.scrapers:Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'C tedra para la Resoluci n de Conflictos, Mediaci n y Transformaci n
Digital'}
2024-04-28 18:52:04 [scrapy.core.scrapers] DEBUG: Scraped from <200
https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.html>
{'Ambito': 'C tedra para la Resoluci n de Conflictos, Mediaci n y Transformaci n
Digital'}
INFO:scrapy.core.engine:Closing spider (finished)
2024-04-28 18:52:04 [scrapy.core.engine] INFO: Closing spider (finished)
INFO:scrapy.statscollectors:Dumping Scrapy stats:
{'downloader/request_bytes': 339,
 'downloader/request_count': 1,
 'downloader/request_method_count/GET': 1,

```

```

'downloader/response_bytes': 74165,
'downloader/response_count': 1,
'downloader/response_status_count/200': 1,
'elapsed_time_seconds': 0.740278,
'finish_reason': 'finished',
'finish_time': datetime.datetime(2024, 4, 28, 18, 52, 4, 748173,
tzinfo=datetime.timezone.utc),
'item_scraped_count': 17,
'log_count/DEBUG': 19,
'log_count/INFO': 10,
'memusage/max': 141746176,
'memusage/startup': 141746176,
'response_received_count': 1,
'scheduler/dequeued': 1,
'scheduler/dequeued/memory': 1,
'scheduler/enqueued': 1,
'scheduler/enqueued/memory': 1,
'start_time': datetime.datetime(2024, 4, 28, 18, 52, 4, 7895,
tzinfo=datetime.timezone.utc)}
2024-04-28 18:52:04 [scrapy.statscollectors] INFO: Dumping Scrapy stats:
{'downloader/request_bytes': 339,
'downloader/request_count': 1,
'downloader/request_method_count/GET': 1,
'downloader/response_bytes': 74165,
'downloader/response_count': 1,
'downloader/response_status_count/200': 1,
'elapsed_time_seconds': 0.740278,
'finish_reason': 'finished',
'finish_time': datetime.datetime(2024, 4, 28, 18, 52, 4, 748173,
tzinfo=datetime.timezone.utc),
'item_scraped_count': 17,
'log_count/DEBUG': 19,
'log_count/INFO': 10,
'memusage/max': 141746176,
'memusage/startup': 141746176,
'response_received_count': 1,
'scheduler/dequeued': 1,
'scheduler/dequeued/memory': 1,
'scheduler/enqueued': 1,
'scheduler/enqueued/memory': 1,
'start_time': datetime.datetime(2024, 4, 28, 18, 52, 4, 7895,
tzinfo=datetime.timezone.utc)}
INFO:scrapy.core.engine:Spider closed (finished)
2024-04-28 18:52:04 [scrapy.core.engine] INFO: Spider closed (finished)

```

L'execució de Scrapy mostra un registre detallat amb tots els esdeveniments que han anat passant, fet que és molt útil per identificar problemes, sobretot en captures complexes. En el nostre cas, a més, podem veure com s'han extret els noms dels diferents àmbits de les titulacions de la UOC.

4.3 Web crawling en Python amb BeautifulSoup

Una altra opció per poder extreure dades de pàgines web a partir dels seus fitxers HTML o XML es utilitzar la llibreria BeautifulSoup. És una eina popular per fer web scraping, ja que té una sintaxis més senzilla que Scrapy.

La biblioteca BeautifulSoup analitza l'HTML (o XML) d'una pàgina web i el converteix en un conjunt d'objectes Python que podeu navegar i manipular fàcilment. Permet cercar, filtrar i modificar els elements HTML d'una manera senzilla i eficient.

A continuació repetirem l'extracció que hem fet a l'apartat anterior amb Scrapy, però amb BeautifulSoup.

Primer de tot, utilitzem la llibreria `requests` per obtenir el contingut HTML de la pàgina web que volem analitzar. A continuació, creem un objecte BeautifulSoup passant el contingut HTML que hem obtingut i un parser (en aquest cas, al tractar-se d'un fitxer HTML especifiquem `html.parser`).

Si mostrem per pantalla el contingut de l'objecte `soup` veurem tota la informació de la pàgina HTML.

```
[19]: # Carreguem les llibreries necessàries
import requests
from bs4 import BeautifulSoup

# Especifiquem la URL
URL = "https://www.uoc.edu/portal/es/universitat/estudis-centres-recerca/index.
      ↪html"

# obtenim la resposta y el contingut de la URL
response = requests.get(URL)
soup = BeautifulSoup(response.content, 'html.parser')

# Descomentar per veure el contingut de soup
#print(soup)
```

```
DEBUG:urllib3.connectionpool:Starting new HTTPS connection (1): www.uoc.edu:443
2024-04-28 18:52:04 [urllib3.connectionpool] DEBUG: Starting new HTTPS
connection (1): www.uoc.edu:443
DEBUG:urllib3.connectionpool:https://www.uoc.edu:443 "GET
/portal/es/universitat/estudis-centres-recerca/index.html HTTP/1.1" 200 None
2024-04-28 18:52:05 [urllib3.connectionpool] DEBUG: https://www.uoc.edu:443 "GET
/portal/es/universitat/estudis-centres-recerca/index.html HTTP/1.1" 200 None
```

Com ja hem vist a l'anterior apartat, la informació que ens interessa està dintre de l'element `h3 card-title`. També sabem que hi ha diferents àmbits que volem recollir, i que tots segueixen la mateixa estructura (el nom es troba en forma de text dintre d'aquest element `h3`). Per això, utilitzarem la funció `find_all` per guardar aquests elements en una llista `ambits`.

Si mostrem el contingut d'aquesta llista, podem veure que cada element conté la informació HTML d'un àmbit.


```
[20]: ambits = soup.find_all('h3', class_='h3 card-title')
      print(ambits[0])
      print(ambits[1])
```

```
<h3 class="h3 card-title" id="card-title-1682057768418"><a
href="/portal/es/estudis_arees/arts_humanitats/index.html"
target="_blank">Estudios de Artes y Humanidades<span aria-hidden="true"
class="absolute-link"></span></a></h3>
<h3 class="h3 card-title" id="card-title-1682057768482"><a
href="/portal/es/estudis_arees/ciencias_informacio_comunicacio/index.html"
target="_blank">Estudios de Ciencias de la Información y de la Comunicación<span
aria-hidden="true" class="absolute-link"></span></a></h3>
```

Però en aquest cas només volem el nom, així que podem iterar sobre la llista i mostrar el text de cada element.

```
[21]: for g in ambits:
      ambit = g.text.strip() # .strip() es fa servir per eliminar espais
      ↪innecessaris
      print(ambit)
```

```
Estudios de Artes y Humanidades
Estudios de Ciencias de la Información y de la Comunicación
Estudios de Ciencias de la Salud
Estudios de Derecho y Ciencia Política
Estudios de Economía y Empresa
Estudios de Informática, Multimedia y Telecomunicación
Estudios de Psicología y Ciencias de la Educación
Internet Interdisciplinary Institute (IN3)
eHealth Center
eLearning Innovation Center
Escuela de Doctorado
Cátedra UNESCO de Educación y Tecnología para el Cambio Social
Cátedra UNESCO de Alimentación, Cultura y Desarrollo
Cátedra UOC-COCEMFE para la Autonomía Personal y Salud Digital
Cátedra Barcelona - UOC en Economía Digital
Cátedra Pau Casals de Música y Defensa de la Paz y de los Derechos Humanos
Cátedra para la Resolución de Conflictos, Mediación y Transformación Digital
Cátedras inactivas
```

5 Exercicis i preguntes teòriques

La part avaluable d'aquesta unitat consisteix en el lliurament d'un fitxer IPython Notebook amb extensió IPYNB que contindrà els diferents exercicis i les preguntes teòriques que s'han de contestar. Trobareu el fitxer (prog_datasci_5_python_entrega.ipynb) amb les activitats a la mateixa carpeta que aquest notebook que esteu llegint.

5.1 Instruccions importants

És molt important que a l'hora de lliurar el fitxer Notebook amb les vostres activitats us assegureu

que:

1. Les vostres solucions siguin originals. Esperem no detectar-hi còpia directa entre estudiants.
2. Tot el codi estigui correctament documentat. El codi sense documentar equivaldrà a un 0.
3. El fitxer comprimit que lliureu és correcte (conté les activitats de la PAC que heu de lliurar).

Per fer el lliurament, heu d'anar a la carpeta del drive Colab Notebooks, clicant botó dret a la PAC en qüestió i fent Download. D'aquesta manera us baixareu la carpeta de la PAC comprimida en zip. Aquest és l'arxiu que heu de pujar al campus de virtual de l'assignatura.

6 Bibliografia

Per obtenir més informació sobre els formats de dades presentats en aquesta unitat, podeu consultar l'w3schools ([JSON](#), [XML](#) i [HTML](#)).

7 Annex: L'API de googlemaps

Aquest annex conté un exemple addicional d'accés a API amb llibreries de Python. En concret, l'exemple mostra com accedir a l'API de googlemaps. En el passat, l'ús d'aquesta API era gratuït, però actualment l'ús de l'API té múltiples restriccions i, tot i que es poden fer algunes peticions gratuïtament, és necessari proporcionar dades de la nostra targeta de crèdit per poder interactuar amb l'API. Podeu revisar el codi d'aquest exemple per tenir un exemple més de l'ús de llibreries per accedir a APIs, o bé crear un compte a la plataforma de google developers i provar els exemples proporcionats. En aquest últim cas, recordeu revisar la política de cobrament de googlemaps, per assegurar que no sobrepassu el límit gratuït, abans de realitzar les proves.

Google maps disposa d'un [conjunt d'API](#) molt populars que permeten, entre d'altres, obtenir les coordenades geogràfiques d'una adreça, aconseguir indicacions per desplaçar-se d'un punt a un altre, o adquirir dades sobre l'elevació del terreny a qualsevol punt del món. La llibreria [googlemaps](#) integra peticions a l'API de Google en codi Python.

Per fer servir les APIs de Google Maps, cal registrar un usuari i obtenir una clau d'autenticació, que adjuntarem a les peticions que es facin contra l'API. A més, haurem d'especificar quines APIs concretes farem servir.

A l'exemple següent, farem aquests tres passos per obtenir la clau d'autenticació:

1. Crearem un projecte a la plataforma de Google Developers.
2. Activarem les APIs desitjades.
3. Sol·licitarem credencials d'accés.

En primer lloc crearem un nou projecte a l'entorn de desenvolupadors de google. Ens dirigirem a: <https://console.developers.google.com/apis/library> i farem clic sobre «Project: New project». Assignarem un nom qualsevol al projecte i confirmarem la creació clicant sobre «Create».

Un cop creat el projecte, activarem les APIs que farem servir. Primer, seleccionarem l'API de geocodificació ([Google Maps Geocoding API](#)), que es troba a la categoria *Google Maps APIs* (és possible que hagueu de prémer sobre el botó «more» per veure la llista completa d' APIs). Farem clic sobre «Enable» per activar-la.

Repetirem el procés per a l'API d'adreces ([Google Maps Directions API](#)), que es troba també a la categoria *Google Maps APIs*.

Finalment, farem clic sobre el menú «Credentials», indicarem «Create credentials» i escollirem «API Key». Ens apareixerà una finestra amb una cadena de caràcters que representa la nostra

clau. Perquè l'exemple següent funcioni, cal que assigneu a la variable `api_key` el valor de la vostra clau.

[22]: `!pip install googlemaps`

```
Collecting googlemaps
  Downloading googlemaps-4.10.0.tar.gz (33 kB)
  Preparing metadata (setup.py) ... done
Requirement already satisfied: requests<3.0,>=2.20.0 in
/usr/local/lib/python3.10/dist-packages (from googlemaps) (2.31.0)
Requirement already satisfied: charset-normalizer<4,>=2 in
/usr/local/lib/python3.10/dist-packages (from requests<3.0,>=2.20.0->googlemaps)
(3.3.2)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.10/dist-
packages (from requests<3.0,>=2.20.0->googlemaps) (3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.10/dist-packages (from requests<3.0,>=2.20.0->googlemaps)
(2.0.7)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.10/dist-packages (from requests<3.0,>=2.20.0->googlemaps)
(2024.2.2)
Building wheels for collected packages: googlemaps
  Building wheel for googlemaps (setup.py) ... done
  Created wheel for googlemaps: filename=googlemaps-4.10.0-py3-none-any.whl
size=40712
sha256=13aae2433f6e6057f85a935501f306e8553dd5ab156b8c30c2dae9c58589b0f7
  Stored in directory: /root/.cache/pip/wheels/17/f8/79/999d5d37118fd35d7219ef57
933eb9d09886c4c4503a800f84
Successfully built googlemaps
Installing collected packages: googlemaps
Successfully installed googlemaps-4.10.0
```

```
[23]: # Importem la llibreria googlemaps, que interactuarà amb l'API de Google Maps.
import googlemaps

# Importem la llibreria datetime, que ens ofereix funcions de maniobres de
↪ dates.
from datetime import datetime

#####
# ATENCIÓ! Assigneu a la variable api_key la clau que hagueu obtingut de Google.
api_key = "AIzaSyDn6wZ-HxkCBQ8szDePoBiKr2drlNsafyA"
#####

# Inicialitzem el client, indicant la clau d'autenticació,
gmaps = googlemaps.Client(key=api_key)
```

INFO:googlemaps.client:API queries_quota: 60

2024-04-28 18:52:19 [googlemaps.client] INFO: API queries_quota: 60

En primer lloc, farem servir l'API de geocodificació per obtenir dades d'una adreça per mitjà del mètode `Geocode` del client de Google Maps que ens ofereix la llibreria (emmagatzemat a la variable `gmaps`).

```
[24]: # Utilitzem l'API de geocodificació per obtenir dades d'una adreça.
geocode_result = gmaps.geocode('Rambla del Poblenou, 156, Barcelona')
print("----- Resultat de geocode -----")
json_print(geocode_result, 20)
```

```
DEBUG:urllib3.connectionpool:Starting new HTTPS connection (1):
maps.googleapis.com:443
2024-04-28 18:52:19 [urllib3.connectionpool] DEBUG: Starting new HTTPS
connection (1): maps.googleapis.com:443
DEBUG:urllib3.connectionpool:https://maps.googleapis.com:443 "GET /maps/api/geoc
ode/json?address=Rambla+del+Poblenou%2C+156%2C+Barcelona&key=AIzaSyDn6wZ-
HxkCBQ8szDePoBiKr2drlNsafyA HTTP/1.1" 200 None
2024-04-28 18:52:19 [urllib3.connectionpool] DEBUG:
https://maps.googleapis.com:443 "GET /maps/api/geocode/json?address=Rambla+del+P
oblenou%2C+156%2C+Barcelona&key=AIzaSyDn6wZ-HxkCBQ8szDePoBiKr2drlNsafyA
HTTP/1.1" 200 None
```

```
----- Resultat de geocode -----
[
  {
    "address_components": [
      {
        "long_name": "156",
        "short_name": "156",
        "types": [
          "street_number"
        ]
      },
      {
        "long_name": "Rambla del Poblenou",
        "short_name": "Rambla del Poblenou",
        "types": [
          "route"
        ]
      },
      {
        "long_name": "Sant Mart\u00ed",
        "short_name": "Sant Mart\u00ed",

```

Un altre exemple de l'ús de l'API de geocodificació utilitza el mètode `reverse_geocode` per obtenir informació sobre unes coordenades geogràfiques concretes:

```
[25]: # Obtenim dades sobre unes coordenades geogràfiques.
reverse_geocode_result = gmaps.reverse_geocode((41.2768089, 1.9884642))
print("----- Resultat de reverse geocode -----")
json_print(reverse_geocode_result, 20)
```

```
DEBUG:urllib3.connectionpool:https://maps.googleapis.com:443 "GET
/maps/api/geocode/json?latlng=41.2768089%2C1.9884642&key=AIzaSyDn6wZ-
HxkCBQ8szDePoBiKr2drlNsafyA HTTP/1.1" 200 None
2024-04-28 18:52:20 [urllib3.connectionpool] DEBUG:
https://maps.googleapis.com:443 "GET
/maps/api/geocode/json?latlng=41.2768089%2C1.9884642&key=AIzaSyDn6wZ-
HxkCBQ8szDePoBiKr2drlNsafyA HTTP/1.1" 200 None

----- Resultat de reverse geocode -----
```

```
[
  {
    "address_components": [
      {
        "long_name": "17",
        "short_name": "17",
        "types": [
          "street_number"
        ]
      },
      {
        "long_name": "Avinguda del Canal Ol\u00ededmpic",
        "short_name": "Av. del Canal Ol\u00ededmpic",
        "types": [
          "route"
        ]
      },
      {
        "long_name": "Castelldefels",
        "short_name": "Castelldefels",

```

L'exemple següent interactua amb l'API d'adreces fent servir el mètode *directions* de la llibreria googlemaps de Python, per obtenir indicacions de desplaçament entre dos punts.

```
[26]: # Obtenim indicacions sobre com anar d'una adreça a una altra, considerant el
↳ trànsit del moment actual.
now = datetime.now()
directions_result = gmaps.directions("Carrer Colom, 114, Terrassa",
                                      "Carrer Sant Antoni, 1, Salt",
                                      mode="transit",
                                      departure_time=now)
print("----- Resultat de directions -----")
json_print(directions_result, 15)
```

```
DEBUG:urllib3.connectionpool:https://maps.googleapis.com:443 "GET /maps/api/dire
ctions/json?departure_time=1714330340&destination=Carrer+Sant+Antoni%2C+1%2C+Sal
t&mode=transit&origin=Carrer+Colom%2C+114%2C+Terrassa&key=AIzaSyDn6wZ-
HxkCBQ8szDePoBiKr2drlNsafyA HTTP/1.1" 200 None
2024-04-28 18:52:20 [urllib3.connectionpool] DEBUG:
https://maps.googleapis.com:443 "GET /maps/api/directions/json?departure_time=17
14330340&destination=Carrer+Sant+Antoni%2C+1%2C+Salt&mode=transit&origin=Carrer+
Colom%2C+114%2C+Terrassa&key=AIzaSyDn6wZ-HxkCBQ8szDePoBiKr2drlNsafyA HTTP/1.1"
200 None
```

----- Resultat de directions -----

```
[
  {
    "bounds": {
      "northeast": {
        "lat": 41.6379081,
        "lng": 2.3566036
      },
      "southwest": {
        "lat": 41.4811223,
        "lng": 2.014278
      }
    },
    "copyrights": "Map data \u00a92024 Inst. Geogr. Nacional",
    "legs": [
      {
        [...]

```

Fixeu-vos que, en aquest cas, no hem hagut de gestionar les peticions HTTP manualment: la llibreria ho ha fet per nosaltres de manera transparent.

A més, les funcions de la llibreria ens tornen directament objectes Python, que es poden fer servir com qualsevol altre. Per exemple, podem seleccionar només una part de les respostes de les API segons el nostre interès:

```
[27]: # Mostrem les claus del diccionari que retorna la crida a geocode.
geocode_result[0].keys()
```

```
[27]: dict_keys(['address_components', 'formatted_address', 'geometry', 'place_id',
'types'])
```

```
[28]: # Mostrem únicament les coordenades geogràfiques de la direcció d'interès.
geocode_result[0]["geometry"]["location"]
```

```
[28]: {'lat': 41.4065936, 'lng': 2.1944287}
```

```
[29]: # Mostrem les localitzacions properes a les coordenades geogràfiques que hem
      ↪preguntat amb 'reverse_geocode',
      # tot imprimint-ne les coordenades exactes i l'adreça.
```

```
for result in reverse_geocode_result:
    print(result["geometry"]["location"], result["formatted_address"])
```

```
{'lat': 41.2772149, 'lng': 1.9892062} Av. del Canal Olímpic, 17, 08860
Castelldefels, Barcelona, Spain
{'lat': 41.2768089, 'lng': 1.9884642} 7XGQ+P9 Castelldefels, Spain
{'lat': 41.2761555, 'lng': 1.9888075} Carrer d'Esteve Terradas, 08860
Castelldefels, Barcelona, Spain
{'lat': 41.2807588, 'lng': 1.9799604} Castelldefels, Barcelona, Spain
{'lat': 41.280735, 'lng': 1.9799522} Municipality of Castelldefels, Barcelona,
Spain
{'lat': 41.2792267, 'lng': 1.9636914} 08860 Castelldefels, Barcelona, Spain
{'lat': 41.3847492, 'lng': 1.949021} Baix Llobregat, Barcelona, Spain
{'lat': 41.7435073, 'lng': 1.9881527} Barcelona, Spain
{'lat': 41.5911589, 'lng': 1.5208624} Catalonia, Spain
{'lat': 40.463667000000001, 'lng': -3.74922} Spain
```

```
[30]: # Mostrem únicament la distància del trajecte entre els dos punts preguntats a
      ↪ l'API d'adreces.
print(directions_result[0]["legs"][0]["distance"])
```

```
{'text': '47.9 km', 'value': 47867}
```

4 Autors

- Autor original **Cristina Pérez Solà**, 2017.
- Actualitzat per **Cristina Pérez Solà**, 2019.
- Actualitzat per **Alexandra Abós Ortega**, 2024.

```
<div style="width:0%;">&nbsp;</div>
```

```
    
</div>
```